



Smart BLE Tag Reminder System For Student

By

Ran Vijay Singh - 24BCE5484

Garima Nidhi - 24BCE1782

Roshni Haldar - 24BCE5019

Janet Pushpa - 24BCE1932

Under the guidance of

Dr Ravi Tiwari

Microprocessor & Microcontroller

VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI-600127

DECLARATION

I hereby declare that this written submission is a representation of my own ideas, expressed in my own words. Wherever I have included ideas or words from other sources, I have appropriately cited and referenced them. I affirm that I have adhered to the principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea, data, fact, or source in this submission. I understand that any violation of these principles may result in disciplinary action by the Institute and could also lead to legal consequences from the original sources if proper citation or necessary permissions have not been obtained.

(Signature)

Ran Vijay Singh

Garima Nidhi

Roshni Halder

Janet Pushpa

CERTIFICATE

It is certified that the work contained in the **MPMC** project titled
“Smart BLE Tag Reminder System for Students” by
**Ran Vijay Singh (Roll No: 24BCE1758), Garima Nidhi (Roll No:
24BCE1782), Roshni Haldar (Roll No: 24BCE5019) and Janet Pushpa
(Roll No: 24BCE1932)** has been carried out under my supervision and that
this work has not been submitted elsewhere for a degree.

Signature of Supervisor Name:

School: SENSE

VIT, Chennai

TABLE OF CONTENTS

- Declaration
- Certificate
- Abstract
- Introduction
- ESP 32
- HM 10 Module
- Working
- Block Diagram
- Code
- Applications

ABSTRACT

This project presents a Smart BLE Tag Reminder System using an ESP32 microcontroller and Bluetooth Low Energy (BLE) technology to help users avoid forgetting important items. The system uses self-built BLE tags created with HM-10 modules, which act as custom BLE emitters placed on essential belongings such as ID cards and bags. Each HM-10 based tag continuously emits a BLE signal that the ESP32 detects, enabling real-time monitoring of tag proximity.

The ESP32 monitors the signal strength of each tag and compares the location of the tagged items relative to a pivot tag placed on a critical item like a bag. When a tagged item moves beyond a set distance from the pivot, the system triggers a buzzer alert to notify the user of a potentially forgotten item. This approach provides a practical wireless solution for personal item tracking to reduce forgetfulness.

This project demonstrates the effective integration of DIY BLE modules with embedded microcontroller logic to implement a reliable smart reminder system. By leveraging Bluetooth Low Energy communication and ESP32's capabilities, it offers an efficient and customizable IoT-based solution for asset monitoring and user notification in everyday life.

INTRODUCTION

This project focuses on designing an efficient and user-friendly smart reminder system using an ESP32 microcontroller and Bluetooth Low Energy (BLE) technology. The system aims to help users avoid forgetting important personal items by continuously monitoring the proximity of BLE tags attached to those items. These tags are self-built using HM-10 BLE modules, which act as custom low-power BLE emitters, making the system customizable and cost-effective.

The core principle of the system relies on the ESP32's ability to scan and detect BLE signals emitted by the HM-10 tags. By analyzing the signal strength, the ESP32 determines whether the tagged items are within a specified range relative to a pivot tag—such as one on a bag or a frequently carried essential. If an item moves beyond the defined range without the pivot tag, the system triggers an alert through a buzzer, notifying the user to prevent forgetting valuable belongings.

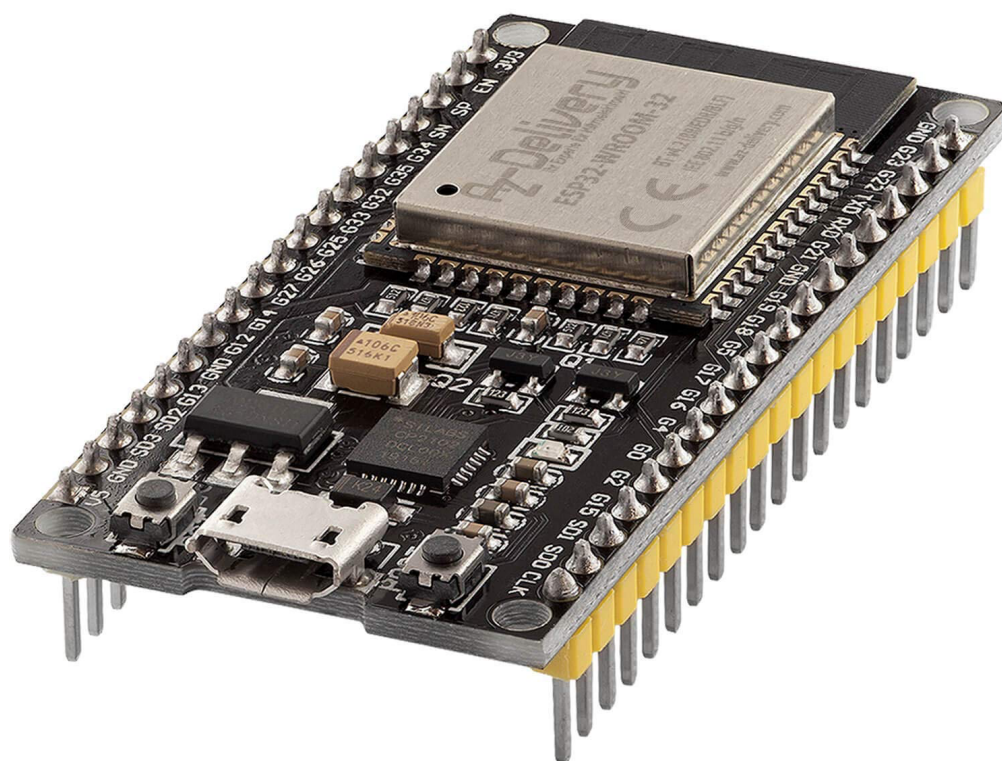
This project demonstrates the practical application of embedded systems and wireless communication technologies for real-world convenience. It highlights how DIY BLE modules can be integrated with microcontroller logic to create a smart, IoT-enabled asset tracking and reminder system suitable for everyday use by students, professionals, and anyone needing efficient item management.

ESP 32

The ESP32 is a powerful and versatile microcontroller module designed for IoT and embedded applications, featuring dual-core processors, integrated Wi-Fi, and Bluetooth Low Energy (BLE) capabilities. It includes multiple digital input/output pins, ADC channels, PWM outputs, and hardware interfaces to connect with sensors, actuators, and communication modules. The ESP32 is widely supported by comprehensive development environments including Arduino IDE and ESP-IDF, enabling easy programming and rapid prototyping.

For this project, the ESP32 serves as the central controller that actively scans for BLE signals emitted by self-built HM-10 BLE tags attached to personal items. Its integrated BLE module allows it to detect the presence and signal strength of nearby tags, making it ideal for real-time proximity monitoring. The ESP32 processes these signals, running decision algorithms to identify whether required items are within range and triggers buzzer or LED alerts if items are detected as forgotten.

The combination of the ESP32's advanced features and BLE support enables the creation of an efficient and customizable smart reminder system. Its connectivity options and low power consumption make it suitable for continuous operation in everyday smart asset tracking, offering a flexible hardware platform that easily integrates with a variety of IoT components for enhanced user convenience and reliability.

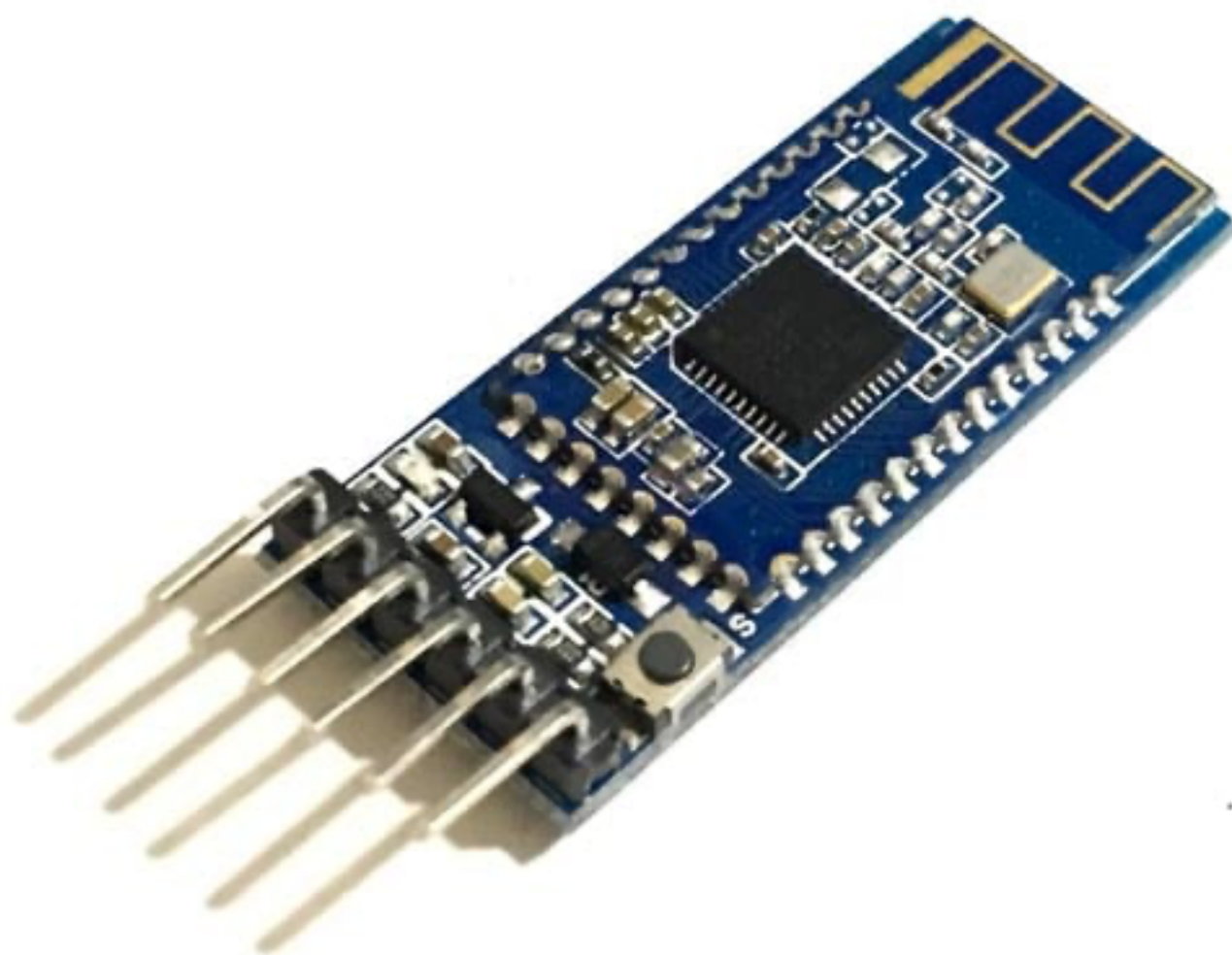


HM 10 Module(BLE Bluetooth Module)

The HM-10 is a compact Bluetooth 4.0 Low Energy (BLE) module based on the Texas Instruments CC2540 or CC2541 chipset, widely used for embedding wireless BLE communication in embedded systems like Arduino, ESP32, and similar microcontrollers. Operating at the 2.4GHz ISM frequency band with Gaussian Frequency Shift Keying (GFSK) modulation, it supports both master and slave roles, enabling versatile wireless connectivity. Powered by a 3.3V source with low power consumption—approximately 8.5mA active current and as low as 50-200µA in sleep mode—the HM-10 is well suited for battery-powered devices aiming for efficient operation.

This module provides seamless, transparent data transmission through a UART serial interface with default 9600 baud rate configurable via AT commands, simplifying integration into various projects. It boasts a communication range up to 100 meters in open space and supports secure connections with authentication and encryption features. The HM-10 is highly configurable via a rich AT command set, allowing customization of device name, baud rate, transmit power, and role (master or slave), making it ideal for smart IoT devices, remote controls, and asset tracking solutions.

Physically compact at approximately 27mm by 13mm in size, the HM-10 integrates well into compact designs, providing robust Bluetooth LE capabilities with ease of use. Its flexibility, low energy consumption, and reliable performance make it a favored choice for wireless communication modules in various embedded systems, enabling projects like your Smart BLE Tag Reminder System to detect proximity and transmit alerts effectively.



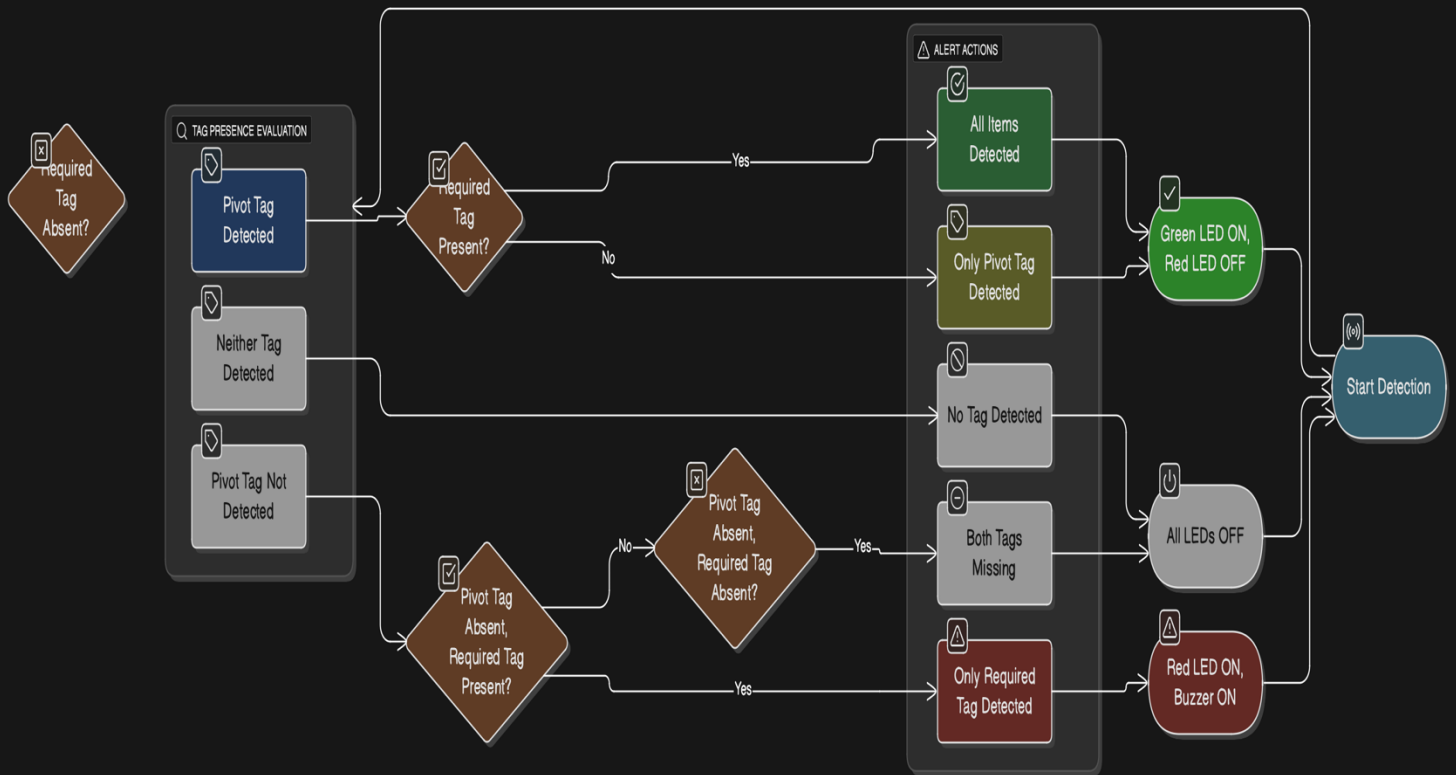
WORKING

The system operates by utilizing the ESP32 microcontroller's integrated Bluetooth Low Energy (BLE) scanning capability to detect BLE signals emitted by self-built HM-10 tags attached to personal items. During operation, the ESP32 continuously scans for nearby BLE devices within a specific timeframe, gathering their MAC addresses and signal strength indicators (RSSI).

The core logic involves identifying a special pivot tag attached to a critical item (like a bag) and other required tags attached to frequently forgotten items. By averaging the RSSI values of detected tags, the ESP32 determines their proximity. When the pivot tag is detected within a predefined RSSI threshold, the system remains silent, indicating all important items are together. However, if the pivot tag is missing but any required tag is still present, the ESP32 triggers a buzzer and LED indicators as alert signals to inform the user of potentially forgotten items.

This approach ensures the user is notified in real time when an important item is left behind relative to the pivot. The system limits buzzer alarm cycles to avoid continuous ringing and resets alarms automatically once all tagged items are proximate again. Thus, it provides an intelligent, wireless reminder solution for everyday personal item tracking based on BLE proximity measurements and embedded control logic.

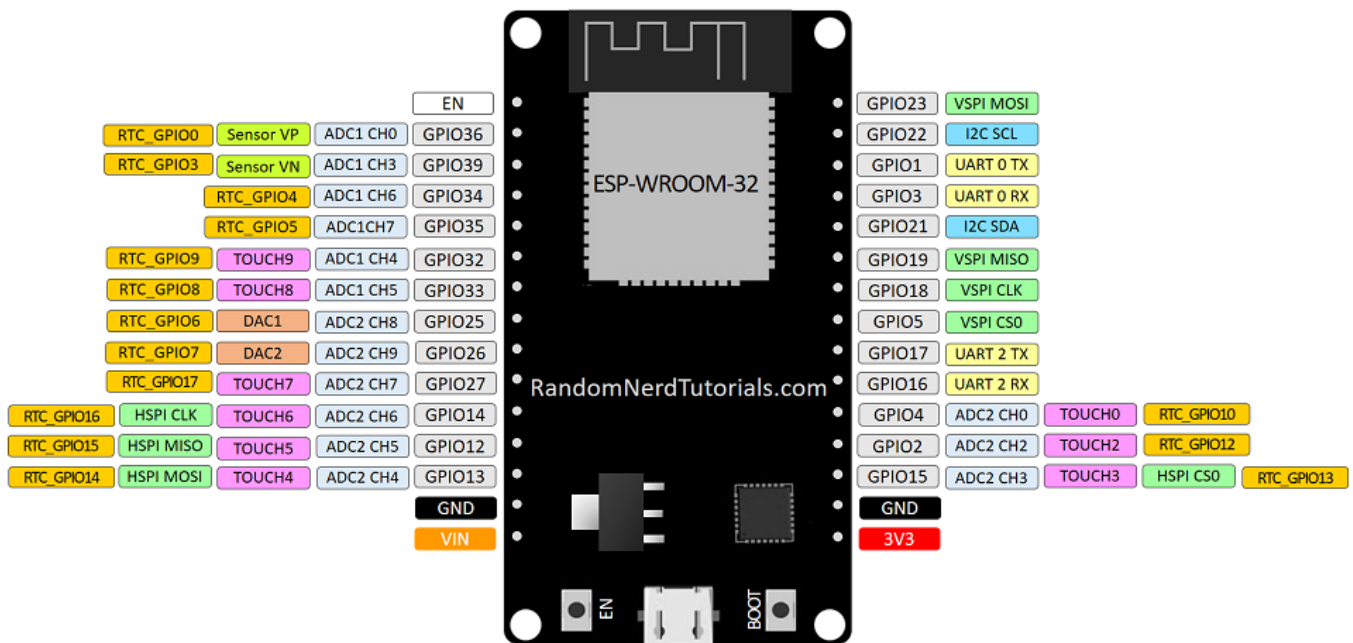
Block Diagram



Pin Diagram & Connections

The ESP32 microcontroller is powered by a regulated 3.3V supply and shares a common ground with the connected components. The buzzer is connected to a digital output pin (generally GPIO 5) on the ESP32, with its other terminal grounded. Similarly, two LEDs are connected with their anodes linked through 330Ω resistors to GPIO 22 (green LED) and GPIO 23 (red LED) respectively, and their cathodes grounded. The HM-10 BLE tags operate wirelessly and do not require direct physical connections to the ESP32; instead, they emit BLE signals detected by the ESP32's built-in BLE scanner. This basic configuration allows the system to scan for the presence of the BLE tags, process their signal strengths, and trigger the buzzer and LEDs for alerts accordingly, all while powered through a stable 3.3V supply rail to ensure reliable operation.

ESP32 DEVKIT V1 – DOIT version with 30 GPIOs



CODE

```
➤ #include <BLEDevice.h>
➤ #include <BLEUtils.h>
➤ #include <BLEScan.h>
➤
➤ #define SCAN_TIME 5
➤ #define BUZZER_PIN 5
➤ #define LED_PIN_RED 23
➤ #define LED_PIN_GREEN 22
➤ #define BUZZER_FREQUENCY 1200
➤ #define BUZZER_DURATION 1000
➤ #define MAX_ALARM_CYCLES 5
➤ #define RSSI_THRESHOLD -80
➤
➤ BLEScan* pBLEScan;
➤
➤ const char* pivotTagMAC = "ff:ff:11:94:ae:5f";
➤ const char* requiredTags[] = {
➤ "ff:ff:11:94:ae:59",
➤ "a4:c1:38:de:ad:be"
➤ };
➤ const int numRequired = 2;
➤
➤ int alarmCycleCount = 0;
➤
➤ // ---- Helper to average RSSI for same MAC ----
➤ int averageRSSI(const char* mac, BLEScanResults* results) {
➤ int sum = 0, count = 0;
➤ for (int i = 0; i < results->getCount(); i++) {
➤ BLEAdvertisedDevice device = results->getDevice(i);
➤ if (strcmp(device.getAddress().toString().c_str(), mac) == 0) {
➤ sum += device.getRSSI();
➤ count++;
➤ }
➤ }
```

```

    return (count > 0) ? (sum / count) : -999; // -999 = not found
}

// ---- Setup ----
void setup() {
    Serial.begin(115200);
    Serial.println("Smart ID Reminder (MAC + RSSI avg) Starting...");

    pinMode(BUZZER_PIN, OUTPUT);
    pinMode(LED_PIN_RED, OUTPUT);
    pinMode(LED_PIN_GREEN, OUTPUT);
    digitalWrite(LED_PIN_RED, HIGH);
    digitalWrite(LED_PIN_GREEN, HIGH);

    BLEDevice::init("");
    pBLEScan = BLEDevice::getScan();
    pBLEScan->setActiveScan(true);
    pBLEScan->setInterval(50);
    pBLEScan->setWindow(49);
}

// ---- Check proximity using avg RSSI ----
bool isTagNearby(const char* mac, BLEScanResults* results) {
    int avgRSSI = averageRSSI(mac, results);
    if (avgRSSI == -999) return false;
    Serial.printf(" → %s avg RSSI: %d dBm\n", mac, avgRSSI);
    return (avgRSSI > RSSI_THRESHOLD);
}

// ---- Main Loop ----
void loop() {
    Serial.println("Scanning for BLE devices...");
    BLEScanResults* foundDevices = pBLEScan->start(SCAN_TIME, false);

    bool pivotNearby = isTagNearby(pivotTagMAC, foundDevices);

    if (pivotNearby) {

```

```

➤ alarmCycleCount = 0;
➤ digitalWrite(LED_PIN_GREEN, HIGH);
➤ digitalWrite(LED_PIN_RED, LOW);
➤ Serial.println("Pivot nearby ✅");
➤ } else {
➤ digitalWrite(LED_PIN_GREEN, LOW);
➤ bool anyRequiredLeft = false;
➤
➤ for (int i = 0; i < numRequired; i++) {
➤ if (isTagNearby(requiredTags[i], foundDevices)) {
➤ anyRequiredLeft = true;
➤ break;
➤ }
➤ }
➤
➤ if (anyRequiredLeft && alarmCycleCount < MAX_ALARM_CYCLES) {
➤ Serial.printf("⚠️ Pivot gone (RSSI below %d), items left! Alarm cycle %d/%d\n", RSSI_THRESHOLD,
    alarmCycleCount + 1, MAX_ALARM_CYCLES);
➤ digitalWrite(LED_PIN_RED, HIGH);
➤ tone(BUZZER_PIN, BUZZER_FREQUENCY, BUZZER_DURATION);
➤ alarmCycleCount++;
➤ }
➤ else if (anyRequiredLeft && alarmCycleCount >= MAX_ALARM_CYCLES) {
➤ Serial.println("⚠️ Alarm silenced after max cycles.");
➤ digitalWrite(LED_PIN_RED, LOW);
➤ }
➤ else {
➤ Serial.println("Pivot gone, all required items taken. ✅");
➤ digitalWrite(LED_PIN_RED, LOW);
➤ alarmCycleCount = 0;
➤ }
➤ }
➤
➤ Serial.println("-----");
➤ delay(100);
➤ }

```


APPLICATIONS

1. **Personal Reminder System for Students:**

- Helps students remember to carry important belongings like wallets, keys, books, or gadgets before leaving dormitories or classrooms.
- Provides immediate alerts to prevent misplacement or forgetting essential items.

2. **Shop and Retail Item Security:**

- Monitors items within a retail environment using BLE tags attached to products.
- Detects unauthorized removal or theft by triggering real-time alerts if items move beyond designated secure areas.

3. **Workplace Asset Protection:**

- Tracks critical tools, ID badges, or laptops within office spaces to ensure security and accountability.
- Generates alerts if valuable items are taken out of authorized zones without permission.

4. **Elderly and Special Needs Assistance:**

- Offers reminders and tracking for personal essentials such as medication containers, glasses cases, or hearing aids.

- Assists caregivers by providing notifications if important items are left behind or misplaced.

5. Smart Inventory Management:

- Tracks inventory in warehouses or storage rooms by tagging stock units and monitoring their presence or movement.
- Helps optimize inventory audits and reduces losses due to misplacement or theft.

6. Travel and Luggage Monitoring:

- Ensures travelers do not forget bags, passports, or tickets by providing alerts before departure or when items are left behind in waiting areas.

7. Event and Conference Access Control:

- Uses BLE tags integrated with access badges to monitor participant movement, ensuring secure entry and exit from event venues or restricted zones.